

ALFIDO TECH

Java Developer Internship

TASK 1: Java Basics & OOP Fundamentals

Submitted By:

Alfishan Khan

Pursuing **B.Tech** – Computer Science Engineering

(Artificial Intelligence & Machine Learning)

Institute of Technology and Management

1. ABSTRACT

Java is one of the most popular object-oriented programming languages used for developing desktop, web, and enterprise applications. This task focuses on understanding the fundamental concepts of Java programming and applying Object-Oriented Programming principles through practical implementation.

In this task, various Java concepts such as conditional statements, loops, arrays, classes, objects, inheritance, polymorphism, and encapsulation were implemented. The programs were developed using Visual Studio Code and Java Development Kit (JDK). Through these implementations, practical knowledge of Java syntax and OOP principles was gained.

The task provided hands-on experience in designing structured programs, organizing code into classes, and applying real-world programming concepts. It also enhanced logical thinking, problem-solving abilities, and understanding of how object-oriented principles contribute to building reusable, maintainable, and efficient software applications.

2. OBJECTIVE

- To understand the fundamentals of Java programming.
 - To learn the use of conditional statements, loops, and arrays.
 - To implement Object-Oriented Programming concepts in Java.
 - To understand classes, objects, inheritance, polymorphism, and encapsulation.
 - To gain practical experience in writing and executing Java programs.
 - To improve logical thinking and problem-solving skills through programming.
-

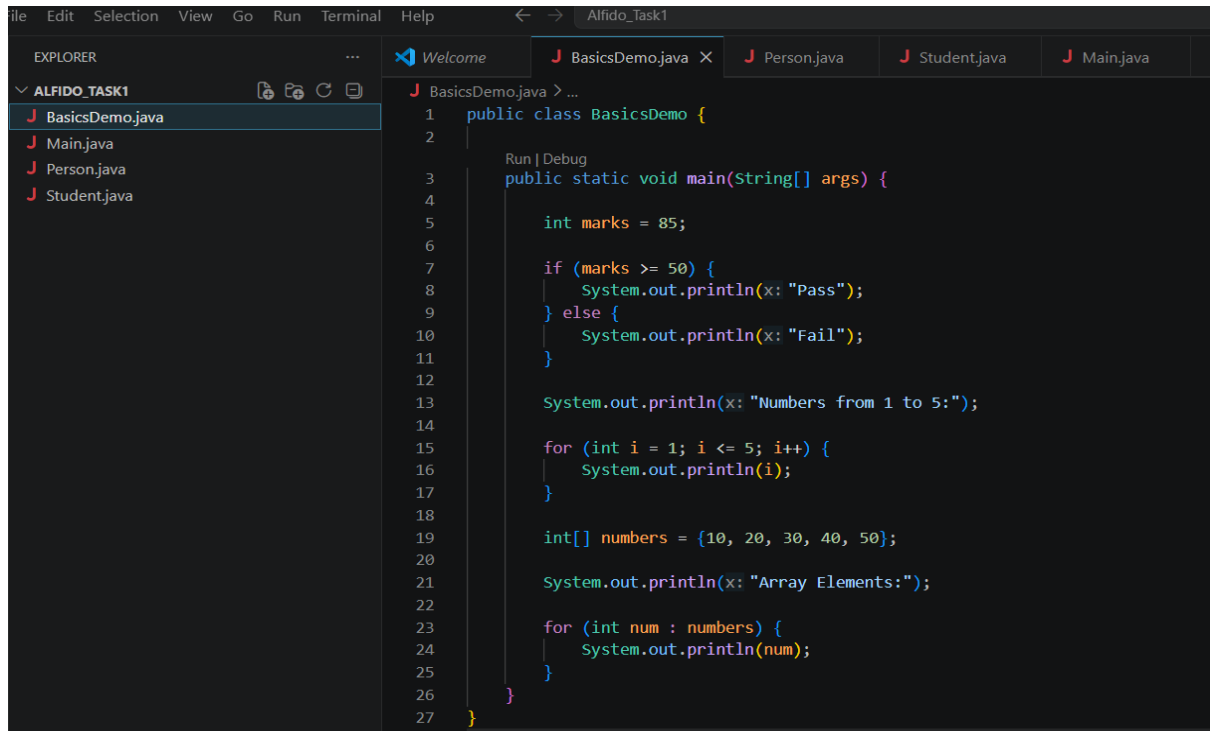
3. SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10/11
 - **Programming Language:** Java
 - **IDE:** Visual Studio Code(VS Code)
 - **JDK Version:** Eclipse Temurin JDK 25
 - **Extensions Used:** Extension Pack for Java
-

4. PROGRAM IMPLEMENTATION

Program 1: BasicsDemo.java

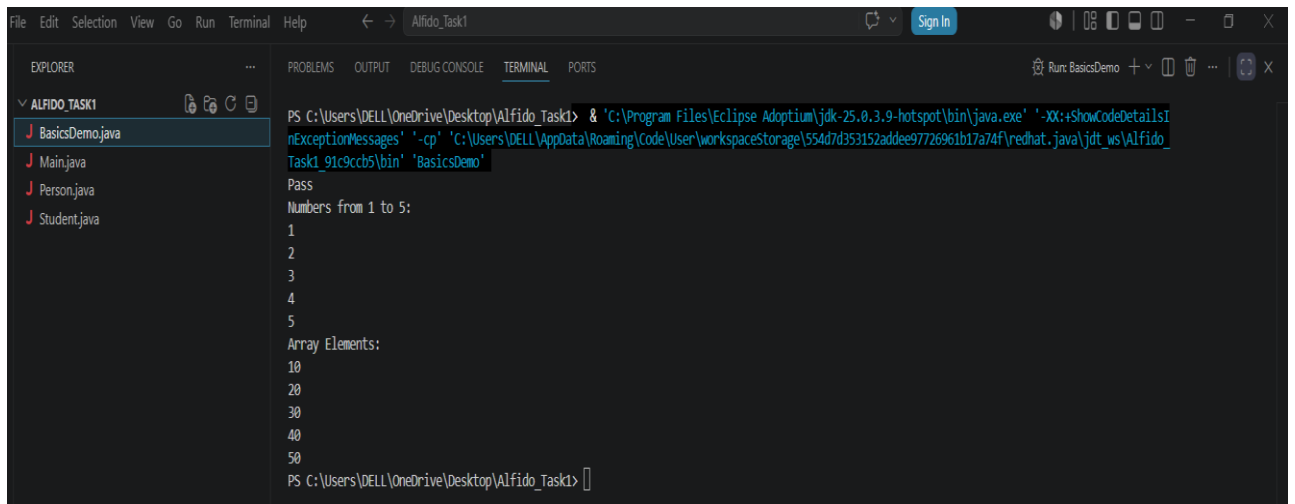
Aim: To demonstrate the implementation of conditional statements, loops, and arrays in Java.

A screenshot of an IDE window titled 'Alfido_Task1'. The Explorer panel on the left shows a project named 'ALFIDO_TASK1' with files 'BasicsDemo.java', 'Main.java', 'Person.java', and 'Student.java'. The BasicsDemo.java file is selected and its code is displayed in the main editor. The code is as follows:

```
1 public class BasicsDemo {  
2  
3     Run | Debug  
4     public static void main(String[] args) {  
5  
6         int marks = 85;  
7  
8         if (marks >= 50) {  
9             System.out.println(x: "Pass");  
10        } else {  
11            System.out.println(x: "Fail");  
12        }  
13  
14        System.out.println(x: "Numbers from 1 to 5:");  
15  
16        for (int i = 1; i <= 5; i++) {  
17            System.out.println(i);  
18        }  
19  
20        int[] numbers = {10, 20, 30, 40, 50};  
21  
22        System.out.println(x: "Array Elements:");  
23  
24        for (int num : numbers) {  
25            System.out.println(num);  
26        }  
27    }  
28 }
```

Figure 1: BasicsDemo.java Code Snippet

Description: The program demonstrates fundamental Java concepts such as conditional statements, loops, and arrays. It first checks whether a student has passed or failed using an if-else statement. Next, it uses a for loop to print numbers from 1 to 5. Finally, it creates an integer array and displays all its elements using an enhanced for loop.



```
File Edit Selection View Go Run Terminal Help
ALFIDO_TASK1
BasicsDemo.java
Main.java
Person.java
Student.java

PS C:\Users\DELL\OneDrive\Desktop\Alfido_Task1> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsI
nExceptionMessages' '-cp' 'C:\Users\DELL\AppData\Roaming\Code\User\workspaceStorage\554d7d353152addee97726961b17a74f\redhat.java\jdt_ws\Alfido
_Task1_91c9ccb5\bin' 'BasicsDemo'
Pass
Numbers from 1 to 5:
1
2
3
4
5
Array Elements:
10
20
30
40
50
PS C:\Users\DELL\OneDrive\Desktop\Alfido_Task1>
```

Figure 2: Output of BasicsDemo.java.

Output Explanation:

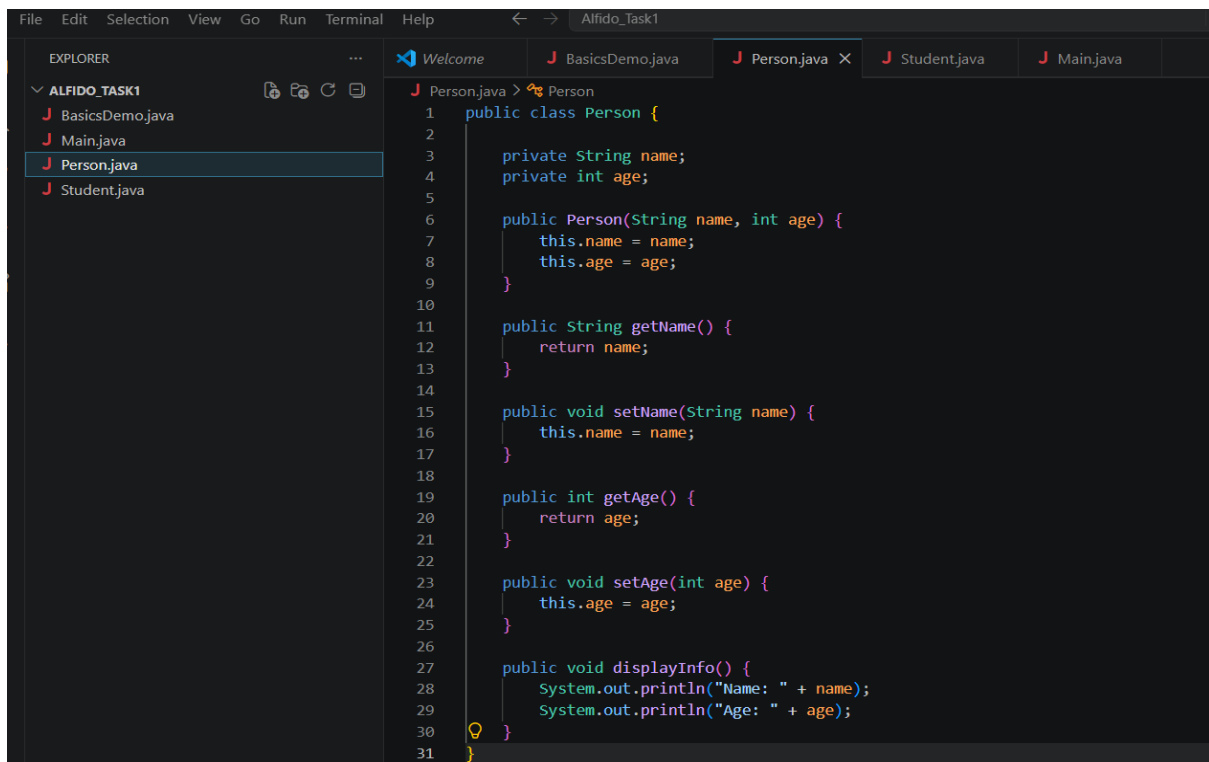
The output first displays "Pass" because the value of marks is 85, which satisfies the condition marks is greater than or equal to 50. Next, the for loop prints the numbers from 1 to 5. Finally, the enhanced for loop traverses the integer array and displays all its elements: 10, 20, 30, 40, and 50. The output confirms the correct implementation of conditional statements, loops, and arrays in Java.

Program 2: OOP Implementation

Files Used:

- Person.java
- Student.java
- Main.java

Aim: To implement Object-Oriented Programming concepts such as classes, objects, inheritance, polymorphism, and encapsulation.

A screenshot of an IDE window titled 'Alfido_Task1'. The Explorer panel on the left shows a project named 'ALFIDO_TASK1' with files 'BasicsDemo.java', 'Main.java', 'Person.java', and 'Student.java'. The 'Person.java' file is selected and its code is displayed in the main editor. The code defines a 'Person' class with private attributes 'name' and 'age', a constructor, and public methods for getting and setting these attributes, as well as a 'displayInfo()' method.

```
1 public class Person {
2
3     private String name;
4     private int age;
5
6     public Person(String name, int age) {
7         this.name = name;
8         this.age = age;
9     }
10
11     public String getName() {
12         return name;
13     }
14
15     public void setName(String name) {
16         this.name = name;
17     }
18
19     public int getAge() {
20         return age;
21     }
22
23     public void setAge(int age) {
24         this.age = age;
25     }
26
27     public void displayInfo() {
28         System.out.println("Name: " + name);
29         System.out.println("Age: " + age);
30     }
31 }
```

Figure 3: Person.java Code Snippet Demonstrating Encapsulation

Description: The Person class demonstrates encapsulation by declaring the data members name and age as private. Public getter and setter methods are provided to access and modify these attributes. The displayInfo() method displays the person's details.

```
1 public class Student extends Person {
2
3     private String course;
4
5     public Student(String name, int age, String course) {
6         super(name, age);
7         this.course = course;
8     }
9
10    @Override
11    public void displayInfo() {
12        System.out.println("Name: " + getName());
13        System.out.println("Age: " + getAge());
14        System.out.println("Course: " + course);
15    }
16 }
```

Figure 4: Student.java Code Snippet

Description: The Student class extends the Person class and introduces an additional attribute named course. It overrides the displayInfo() method to display the student's name, age, and course information, demonstrating inheritance and method overriding.

```
1 public class Main {
2
3     Run | Debug
4     public static void main(String[] args) {
5
6         Person person =
7             new Student(name: "Rahul", age: 20, course: "Java Programming");
8         person.displayInfo();
9     }
10 }
```

Figure 5: Main.java Code Snippet

Description: The Main class acts as the entry point of the application. It creates a Student object using a Person reference and calls the displayInfo() method. This demonstrates runtime polymorphism, where the overridden method of the child class is executed through a parent class reference.

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         Person person =
6             new Student(name: "Rahul", age: 20, course: "Java Programming");
7
8         person.displayInfo();
9     }
10 }
```

```
PS C:\Users\DELL\OneDrive\Desktop\Alfido_Task1> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsI
nExceptionMessages" "-cp" "C:\Users\DELL\AppData\Roaming\Code\User\workspaceStorage\554d7d353152addee97726961b17a74f\redhat.java\jdt_ws\Alfido_
Task1_91c9ccb5\bin" "Main"
Name: Rahul
Age: 20
Course: Java Programming
PS C:\Users\DELL\OneDrive\Desktop\Alfido_Task1>
```

Figure 6: Output of Main.java (OOP Demonstration)

Output Explanation: The output displays the student's Name, Age, and Course details. The displayInfo() method of the Student class is executed through a Person reference object, demonstrating runtime polymorphism. The output also confirms the successful implementation of encapsulation, inheritance, method overriding, and polymorphism in Java.

5. OBJECT-ORIENTED PROGRAMMING CONCEPTS

➤ Class:

A class is a blueprint used to create objects. It contains data members and methods that define the properties and behavior of objects. Classes help in organizing code and making programs easier to manage and maintain. In this task, *Person* and *Student* are classes.

➤ Object:

An object is an instance of a class that represents a real-world entity. Objects are created to access the properties and methods defined inside a class. Multiple objects can be created from a single class, each having its own values. In this task, a *Student* object was created in *Main.java*.

➤ Inheritance:

Inheritance allows one class to acquire the properties and methods of another class. It promotes code reusability and reduces code duplication. In this task, *Student* inherits from *Person*.

➤ Polymorphism:

Polymorphism means one interface with multiple forms. It allows the same method to behave differently depending on the object calling it. This task demonstrates runtime polymorphism through method overriding.

➤ Encapsulation:

Encapsulation is the process of binding data and methods into a single unit and restricting direct access to data. It is achieved by declaring variables as private and accessing them through getter and setter methods.

➤ Advantages of OOP:

- Improves code reusability through inheritance.
- Enhances security through encapsulation.
- Makes programs easier to maintain and update.
- Reduces code duplication and increases efficiency.
- Improves code organization and readability.
- Supports real-world modeling of applications using classes and objects.

6. LEARNING OUTCOMES

- Learned Java syntax and programming fundamentals.
 - Understood conditional statements, loops, and arrays.
 - Implemented classes and objects.
 - Applied inheritance, polymorphism, and encapsulation.
 - Improved problem-solving and logical thinking skills.
 - Gained practical experience in developing Java applications.
-

7. CONCLUSION

Through this task, I gained a strong understanding of Java programming fundamentals and Object-Oriented Programming concepts. I successfully implemented programs using conditional statements, loops, arrays, classes, objects, inheritance, polymorphism, and encapsulation.

The practical implementation of these concepts improved my logical thinking, problem-solving abilities, and coding skills. I also learned how to organize programs into reusable and maintainable classes using OOP principles. The knowledge gained from this task will serve as a foundation for building more advanced java applications in the future.

Overall, this task provided valuable hands-on experience with Java programming and established a solid foundation for developing more advanced Java applications and software systems in the future.

8. REFERENCES

- Oracle Java Documentation
 - Java SE API Documentation
 - Visual Studio Code Documentation
 - Eclipse Temurin JDK Documentation
-